
GraphDTA - Replication

Jerome Rodrigo

July 23, 2025

ABSTRACT

This paper presents a replication study of the GraphDTA model, which integrates Graph Neural Networks (GNNs) and Convolutional Neural Networks (CNNs) to predict drug-target binding affinity. The primary goal is to validate the original study's findings while deepening my understanding of its computational and mathematical underpinnings. I analyze the model architecture, training process, and dataset used, and explore the theoretical foundations of GNNs and CNNs in the context of bioinformatics. Through this replication, I aim to enhance my knowledge in drug discovery, bioinformatics, and machine learning.

Keywords. Graph Neural Networks · Convolutional Neural Networks · Drug-Target Binding Affinity · Deep Learning · Bioinformatics · Drug Discovery · GraphDTA

1 Introduction

This paper is a math-first walkthrough of reimplementing GraphDTA (Nguyen et al.) to learn how deep learning models can predict drug-target binding affinity. Instead of summarizing results from the original work, I rebuild the full pipeline, tokenizing protein sequences, representing small molecules as graphs, encoding them with CNNs and Graph Neural Networks, and combining the learned embeddings for regression. Along the way I unpack the math behind 1D convolutions, global pooling, message passing on molecular graphs, and the loss/objective used for affinity prediction.

2 High Level Overview

dadada

3 Theorems

4 Protein encoding

4.1 One Hot Encoding

So we use one-hot encoding to represent amino acids in protein sequences. How the function works is that it takes a sequence of amino acids and converts each amino acid into a one-hot encoded vector. The amino acids are represented by a number in a dictionary corresponding to their index in the amino acid vocabulary. The one-hot encoded vector is length 22, where the index corresponding to the amino acid is set to 1, and all other indices are set to 0.

4.1.1 How and why one-hot encoding works

So one-hot encoding is a way to represent categorical data as binary vectors. Each category is represented by a vector where one element is set to 1 (the “hot” part/the category that is

being represented) and all other elements in the vector are set to 0. It works for neural networks because it allows the model to learn relationships between different categories without assuming any ordinal relationship between them. In the context of protein sequences, one-hot encoding allows us to represent each amino acid as a unique vector, which can then be processed by neural networks. One-hot encoding works for amino acids really well as the feature space is small (22 amino acids) and the relationships between them are not ordinal.

5 Drug encoding

5.1 SMILES to Graph representation

5.1.1 Atoms to Nodes

Following the paper I used RDKit to convert SMILES compounds into molecular graphs. Each node in the graph is represented by a vector of features with most being one-hot encoded. The features include the atom symbol, the number of adjacent hydrogens, the number of adjacent atoms, the implicit value of the atom, and whether the atom is in a aromatic structure.

5.1.2 Bonds to Edges

Each edge in the graph is made by taking the bonds using RDKit. The bond notes the atom indices and the graph is undirected by making edges both ways. Also the matrix is then tranposed to be used in the GNNs.

6 Graph Neural Networks

6.1 Basic Overview

So Graph Neural Networks (GNNs) work similarly to regular neural networks, but they are built for graph-structured data. They learn to understand relationships between nodes in a graph through encoding neighboring node information. Pretty much each layer in the GNN gets information from neighboring nodes and uses it (e.g., by summing or averaging) to update the given node’s representation through a shared neural transformation. This is done through a process called message passing, where each node sends and receives messages from its neighbors. The GNN learns to aggregate these messages and update the node representations iteratively. This allows the model to learn rich structure-aware embeddings, which is perfect for tasks like drug-target binding affinity prediction where the relationships between atoms in a molecule are crucial.

6.2 GCNs

So the first approach in the GraphDTA paper is to use a Graph Convolutional Network (GCN). GCNs are a type of GNN that applies convolutional operations on graph-structured data.

We represent each molecule as an undirected graph $G = (V, E)$ with:

- $N = |V|$ atoms (nodes),
- Node features $X \in \mathbb{R}^{N \times C}$ Where N is the number of atoms and C is the number of features per atoms (13).
- Adjacency matrix $A \in \{0, 1\}^{N \times N}$ where $A_{\{i,j\}} = 1$ if atoms i and j are bonded.
- Then we have our Degree matrix $D \in \mathbb{R}^{N \times N}$ where $D_{\{i,i\}} = \sum_{\{j=1\}}^{\{N\}} A_{\{i,j\}}$ is the degree of node i which is the number of connections a node has plus itself.
- The normalized weights matrix is $S_{ij} = \frac{1}{\sqrt{D_{ii}D_{jj}}}$ if $A_{ij} = 1$ else $S_{ij} = 0$.

- The reason the we used a normalized weight matrix is to prevent the model from being biased towards nodes with high degrees. This helps the model learn more balanced representations of nodes in the graph.